

Ejercicio 5: Integración y Conexión a Base de Datos con JDBC

Objetivo: Configurar un entorno de base de datos local y utilizar la API JDBC (Java Database Connectivity) para establecer una conexión y realizar las operaciones fundamentales de inserción, actualización y borrado (Insert, Update, Delete).

Paso 1: Instalación de MySQL y MySQL Workbench (Preparación del Entorno)

1. Descarga el **MySQL Installer** desde la página oficial de MySQL.
2. Durante el proceso, asegúrate de instalar **MySQL Server** (el motor de la base de datos) y **MySQL Workbench** (la interfaz gráfica para administrarla).
3. En la configuración del servidor, define una contraseña segura para el usuario administrador (root) y finaliza la instalación.

Paso 2: Creación de la base de datos y la tabla

1. Abre MySQL Workbench y conéctate a tu instancia local utilizando la contraseña del usuario root.
2. Abre una nueva pestaña de consulta (Query) y ejecuta el siguiente script SQL para crear una base de datos y una tabla con 3 campos (id, nombre y departamento):

```
CREATE DATABASE empresa_db;
USE empresa_db;

CREATE TABLE empleados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    departamento VARCHAR(50) NOT NULL
);
```

Paso 3: Clase para establecer la Conexión

Antes de escribir el código Java, debes descargar el **MySQL JDBC Driver (Connector/J)** y agregarlo a las librerías (Classpath) de tu proyecto en tu IDE (como NetBeans o Eclipse).

Crea una clase llamada **ConexionDB** que se encargue exclusivamente de utilizar la clase **DriverManager** para conectar tu aplicación con MySQL:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionDB {
    // Configuración de la URL de conexión, usuario y contraseña
    private static final String URL = "jdbc:mysql://localhost:3306/empresa_db";
    private static final String USER = "root";
    private static final String PASSWORD = "tu_contrasena_aqui"; // Sustituye por tu clave real

    public static Connection conectar() {
        Connection conexion = null;
        try {
            // Estableciendo la conexión utilizando DriverManager
            conexion = DriverManager.getConnection(URL, USER, PASSWORD);
            System.out.println("¡Conexión a la base de datos establecida con éxito!");
        } catch (SQLException e) {
            System.out.println("Error al conectar con la base de datos: " + e.getMessage());
        }
        return conexion;
    }
}
```

Paso 4: Clase para ejecutar el CRUD (Insert, Update, Delete)

Ahora, crea una segunda clase llamada `OperacionesDB` que utilice la conexión para enviar instrucciones SQL a la tabla `empleados`:

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class OperacionesDB {

    // Método para agregar información (INSERT)
    public void insertarEmpleado(String nombre, String departamento) {
        String sql = "INSERT INTO empleados (nombre, departamento) VALUES (?, ?)";

        try (Connection conn = ConexionDB.conectar();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, nombre);
            pstmt.setString(2, departamento);
            pstmt.executeUpdate();
            System.out.println("Empleado '" + nombre + "' insertado correctamente.");

        } catch (SQLException e) {
            System.out.println("Error al insertar: " + e.getMessage());
        }
    }

    // Método para modificar información (UPDATE)
    public void actualizarDepartamento(int id, String nuevoDepartamento) {
        String sql = "UPDATE empleados SET departamento = ? WHERE id = ?";

        try (Connection conn = ConexionDB.conectar();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, nuevoDepartamento);
            pstmt.setInt(2, id);

            int filasAfectadas = pstmt.executeUpdate();
            if (filasAfectadas > 0) {
                System.out.println("El departamento del empleado con ID " + id + " fue actualizado a: " +
nuevoDepartamento);
            } else {
                System.out.println("No se encontró un empleado con el ID: " + id);
            }

        } catch (SQLException e) {
            System.out.println("Error al actualizar: " + e.getMessage());
        }
    }

    // Método para borrar información (DELETE)
    public void borrarEmpleado(int id) {
        String sql = "DELETE FROM empleados WHERE id = ?";

        try (Connection conn = ConexionDB.conectar();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, id);

            int filasAfectadas = pstmt.executeUpdate();
            if (filasAfectadas > 0) {
                System.out.println("El empleado con ID " + id + " ha sido borrado exitosamente.");
            } else {
                System.out.println("No se encontró un empleado con el ID: " + id);
            }

        } catch (SQLException e) {
            System.out.println("Error al borrar: " + e.getMessage());
        }
    }

    // Método principal para poner a prueba todas las funciones
    public static void main(String[] args) {
        OperacionesDB ops = new OperacionesDB();

        // 1. Ejecutar un INSERT
        ops.insertarEmpleado("Ana García", "Recursos Humanos");

        // 2. Ejecutar un UPDATE (Suponiendo que a Ana se le asignó el ID 1)
        ops.actualizarDepartamento(1, "Finanzas");
    }
}
```

```

    // 3. Ejecutar un DELETE
    ops.borrarEmpleado(1);
}
}

```

Ejercicio 5 Actualizado: Arquitectura MVC y Patrón DAO para MySQL

En lugar de mezclar la conexión y las consultas en una sola clase, las nuevas metodologías indican que debemos separar la lógica de negocio y los datos.

Objetivo: Crear una estructura orientada a objetos para manejar la tabla `empleados` utilizando el patrón DAO.

1. **El Modelo (Clase Empleado):** Crea una clase pura de Java (POJO) que represente la tabla.
2. **La Interfaz de Operaciones:** Crea una interfaz `OperacionesDAO` para estandarizar el código de acceso a datos.
3. **La Clase DAO (EmpleadoDAO):** Implementa la interfaz anterior. Aquí utilizarás `PreparedStatement` para ejecutar los `INSERT`, `UPDATE` y `DELETE` de forma segura. El método `consultarTodos()` debe ejecutar un `SELECT *`, iterar el `ResultSet` y devolver un `ArrayList<Empleado>`.

Ejercicio 6 Actualizado: Interacción Avanzada con JTable (Eventos de Ratón y Update/Delete)

Este ejercicio actualiza la vista visual de la tabla, permitiendo seleccionar registros con el ratón para rellenar un formulario lateral y poder modificarlos o borrarlos.

Objetivo: Desarrollar un formulario interactivo (la "Vista" en MVC) que consuma los datos de `EmpleadoDAO` y permita la edición directa.

1. **Diseño de la Interfaz:** Crea un `JFrame` que contenga un contenedor `JScrollPane` con tu `JTable` en el lado derecho, y cajas de texto (`TextField` para ID, Nombre y Departamento) junto con los botones "Modificar" y "Eliminar" en el lado izquierdo.
2. **Poblar la Tabla:** Al cargar la ventana, instancia `EmpleadoDAO`, invoca `consultarTodos()`, itera el `ArrayList` resultante y usa `modeloTabla.addRow()` para llenar el `DefaultTableModel` asociado a tu `JTable`.
3. **Evento de Selección (MouseListener):** Añade un evento de clic del ratón a la tabla (`addMouseListener`). Dentro del evento, obtén la fila seleccionada usando `tabla.getSelectedRow()`.
4. **Transferencia de Datos:** Utiliza el método `tabla.getValueAt(fila, columna)` para extraer el ID, Nombre y Departamento de la fila clickeada y pásalos a los `TextField` correspondientes (`setText()`).
5. **Botón Modificar (UPDATE):** Al pulsar "Modificar", recoge los nuevos valores de los `TextField`, crea un objeto `Empleado`, pásalo al método `modificar(emp)` del DAO, y si tiene éxito, refresca la tabla limpiando el modelo (`setRowCount(0)`) y volviendo a cargar los datos.
6. **Botón Eliminar (DELETE):** Al pulsar "Eliminar", recoge únicamente el ID seleccionado, pásalo al método `eliminar(id)` del DAO y, tras el borrado, refresca visualmente la tabla y limpia las cajas de texto.

Nuevo Ejercicio 7: CRUD Definitivo con Componentes Complejos y Claves Foráneas

Para acercar la aplicación a un entorno real, las fuentes muestran cómo incorporar datos relacionales y componentes multimedia en los formularios.

Objetivo: Integrar listas desplegables y selectores de archivos en el ciclo de guardado de datos.

1. **Preparación de BD (Relaciones):** Modifica la base de datos creando una tabla independiente departamentos (con `id_depto` y `nombre_depto`). En la tabla `empleados`, cambia el campo texto de departamento por una clave foránea que apunte al ID del departamento.
2. **Uso de JComboBox:** Reemplaza el `TextField` del departamento en tu formulario por un `JComboBox`. En la inicialización, consulta la tabla departamentos y llena el desplegable con los nombres. Al momento de guardar o modificar, deberás extraer el índice o valor clave asociado al texto seleccionado para insertarlo en la base de datos de empleados.
3. **Adjuntar Fotografía con JFileChooser:** Añade un botón "Buscar Foto" y un `JLabel` vacío para previsualización. Usa `JFileChooser` para abrir un cuadro de diálogo del sistema operativo que permita buscar archivos de imagen. Captura la ruta del archivo seleccionado, conviértelo a bytes (usando `FileInputStream`) o guárdalo como ruta en texto (`String`) en la base de datos, y utiliza `setIcon()` en el `JLabel` para previsualizar la fotografía del empleado en la aplicación.